

what model parallelism is:

=> it splits up the parameters across GPUs (like zero 3)

=> But communicate activations while zero 3 sends params)

we cover two different types of parallelism.

1: pipeline parallel: layer-wise parallel. + overlapped comp/communication

2: Tensor parallel (+ sequence parallel)

Just two different ways of cutting up the model.

← model is split into layers / data batch is split into micro-batch.

pipeline parallel: process 'micro-batches', send off the first.

micro batch and start computing the second.

save memory
memory-wise

scale

good communications properties, depending on activations

DP: for high throughput.

TP: for splitting within layers

PP: for splitting across layers with micro-batched inputs

→ for memory saving

Trading communication bandwidth for utilization

'zero bubble' pipelining => split up backwards into two parts

1: Backpropagating activations (z, x)

2: computing weight gradients (w)

$\Rightarrow$ pipeline parallel is cutting up along depth, what about width?

$\Rightarrow$ Tensor ~~para~~ parallel: decompose into submatrices,
 $\Downarrow$
add partial sum

sub-matrices

$\Rightarrow$ on GPUs, TP within a node (8 GPUs) due to high-speed interconnects

TP

pros: no bubble, no waiting for others $\Leftarrow$ fast network.

low complexity $\rightarrow$ wrap models without major infra changes
doesn't need large batch size

Cons: $\gg$ much larger communication than PP

inputs memory

gradient memory

optimizer state memory

activation memory $\Rightarrow$ final complexity

per layer

$\Rightarrow$ Tensor parallel + sequence parallel: ^{early} full linear scaling for memory

